DESARROLLO WEB BACKEND

UNIDAD 3

Programación y Desarrollo de Aplicaciones Web con Frameworks Backend

Material de Estudio Teórico

CONTENIDO DE LA UNIDAD

3.1 Servidores web
- 3.1.1 Funcionamiento de un servidor web
- 3.1.2 Configuración básica

3.2 Ciclo de vida de una solicitud web

3.3 Variables de entorno y configuración del proyecto
- 3.3.1 Archivos de configuración (.env)
- 3.3.2 Gestión de entornos: desarrollo, pruebas y producción

3.4 Frameworks MVC backend
- 3.4.1 Introducción al patrón MVC
- 3.4.2 Laravel: estructura y configuración
- 3.4.3 Node.js con Express: estructura y configuración
- 3.4.4 Enrutamiento y controladores

3.5 Lógica de negocio y manejo de sesiones
- 3.5.1 Variables de sesión
- 3.5.2 Cookies y autenticación básica

3.6 Motor de plantillas
- 3.6.1 Blade (Laravel)
- 3.6.2 EJS / Handlebars (Node.js)

3.7 Integración con bases de datos
- 3.7.1 Conexión a la base de datos
- 3.7.2 ORM: concepto y ventajas
- 3.7.3 Eloquent ORM y Sequelize
- 3.7.4 Migraciones y seeders
- 3.7.5 Relaciones entre modelos

3.8 Patrones de diseño aplicados al backend
- 3.8.1 Patrón Repository
- 3.8.2 Service Layer

3.9 CRUD completo
- 3.9.1 Integración de vistas, lógica y BD
- 3.9.2 Validación y manejo de errores
- 3.9.3 Logging y depuración

COMPETENCIA DE LA UNIDAD

Desarrolla aplicaciones web del lado del servidor utilizando frameworks backend (Laravel y Express), implementando el patrón MVC, integración con bases de datos mediante ORM, manejo de sesiones, motores de plantillas y patrones de diseño, para construir sistemas CRUD completos con validación, manejo de errores y buenas prácticas de desarrollo.

3.1 Servidores web

Un servidor web es un software (y, por extensión, el hardware que lo ejecuta) cuya función principal es recibir solicitudes de clientes —generalmente navegadores— a través del protocolo HTTP/HTTPS y responder con los recursos solicitados: páginas HTML, archivos CSS, imágenes, datos JSON o cualquier otro contenido.

En el contexto del desarrollo backend, comprender cómo funciona un servidor web es fundamental porque toda aplicación web se ejecuta sobre uno. Frameworks como Laravel utilizan servidores integrados para desarrollo (php artisan serve) y servidores de producción como Nginx o Apache. Express.js, por su parte, implementa su propio servidor HTTP dentro de Node.js.

3.1.1 Funcionamiento de un servidor web

El funcionamiento de un servidor web sigue un ciclo bien definido que se repite con cada solicitud:

1. Escucha: el servidor se enlaza a un puerto (generalmente 80 para HTTP o 443 para HTTPS) y espera conexiones entrantes.
2. Recepción: cuando un cliente envía una solicitud HTTP (GET, POST, PUT, DELETE, etc.), el servidor la recibe y la analiza, extrayendo el método, la URL, los encabezados y el cuerpo.
3. Procesamiento: el servidor determina qué recurso o lógica corresponde a la URL solicitada. Puede servir un archivo estático directamente o delegar a una aplicación backend (PHP, Node.js, Python, etc.).
4. Respuesta: el servidor construye una respuesta HTTP con un código de estado (200 OK, 404 Not Found, 500 Internal Server Error), encabezados y un cuerpo, y la envía de vuelta al cliente.
5. Cierre o persistencia: dependiendo de la versión de HTTP y los encabezados, la conexión se cierra (HTTP/1.0) o se mantiene abierta para solicitudes adicionales (keep-alive en HTTP/1.1).

DEFINICIÓN

Un servidor web es un programa que implementa el protocolo HTTP para recibir solicitudes y enviar respuestas. Actúa como intermediario entre el cliente (navegador) y la aplicación backend que genera el contenido dinámico.

Servidor	Lenguaje	Características principales
Apache HTTP	C	Modular, .htaccess, mod_rewrite, amplia comunidad
Nginx	C	Alto rendimiento, proxy inverso, bajo consumo de memoria
PHP built-in	PHP	php -S localhost:8000, solo para desarrollo
Node.js HTTP	JavaScript	Servidor integrado en el runtime, base de Express
Caddy	Go	HTTPS automático, configuración sencilla

Servidor HTTP básico en Node.js

```
const http = require('http');

const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/html' });
  res.end('<h1>Hola desde Node.js</h1>');
});

server.listen(3000, () => {
  console.log('Servidor en http://localhost:3000');
});
```

3.1.2 Configuración básica

La configuración de un servidor web varía según el software utilizado, pero existen conceptos comunes a todos:

- Puerto de escucha: define en qué puerto TCP el servidor acepta conexiones (80, 443, 3000, 8000, etc.).
- Document root: directorio base desde donde se sirven los archivos estáticos (en Laravel es /public, en Express se configura con express.static()).
- Virtual hosts: permiten alojar múltiples sitios en un mismo servidor, diferenciándolos por nombre de dominio.
- Módulos y middleware: extensiones que agregan funcionalidades como compresión gzip, CORS, logging o autenticación.
- HTTPS/TLS: configuración de certificados SSL para conexiones seguras.

Configuración de Nginx como proxy inverso para Laravel

```
server {
    listen 80;
    server_name miapp.com;
    root /var/www/miapp/public;

    index index.php index.html;

    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }

    location ~ \.php$ {
        fastcgi_pass unix:/var/run/php/php8.3-fpm.sock;
        fastcgi_param SCRIPT_FILENAME $realpath_root$fastcgi_script_name;
        include fastcgi_params;
    }
}
```

NOTA

En desarrollo, tanto Laravel (php artisan serve) como Express (node app.js) incluyen servidores integrados. En producción, se recomienda un servidor dedicado como Nginx delante de la aplicación, actuando como proxy inverso para gestionar SSL, balanceo de carga y archivos estáticos.

3.2 Ciclo de vida de una solicitud web

Cuando un usuario interactúa con una aplicación web, por ejemplo, haciendo clic en un enlace o enviando un formulario, se desencadena un proceso complejo que involucra múltiples capas de tecnología. Entender este ciclo es crucial para diagnosticar problemas, optimizar el rendimiento y diseñar aplicaciones robustas.

El ciclo completo se puede descomponer en las siguientes fases:

6. Resolución DNS: el navegador traduce el nombre de dominio (ej. miapp.com) a una dirección IP consultando servidores DNS.
7. Conexión TCP: se establece una conexión TCP con el servidor mediante el three-way handshake (SYN → SYN-ACK → ACK).
8. Negociación TLS (si es HTTPS): cliente y servidor intercambian certificados y acuerdan claves de cifrado.
9. Envío de la solicitud HTTP: el navegador envía la solicitud con método, URL, encabezados (Host, User-Agent, Accept, cookies) y opcionalmente un cuerpo.
10. Procesamiento en el servidor: la solicitud pasa por el pipeline del framework — middleware → enrutador → controlador → modelo → vista.
11. Generación de la respuesta: el servidor construye la respuesta HTTP con código de estado, encabezados (Content-Type, Set-Cookie, Cache-Control) y cuerpo (HTML, JSON).
12. Renderizado en el cliente: el navegador interpreta el HTML, solicita recursos adicionales (CSS, JS, imágenes) y renderiza la página.

> **EJEMPLO**
> Cuando un usuario visita http://miapp.com/tareas, el navegador resuelve el DNS, establece la conexión TCP, envía GET /tareas, el servidor ejecuta el middleware de autenticación, el router dirige al TaskController@index, que consulta la base de datos via Eloquent, pasa los datos a la vista Blade tasks/index.blade.php, y devuelve el HTML renderizado con código 200.

En Laravel, el ciclo interno del framework sigue esta secuencia:

13. Entrada (public/index.php): punto de entrada que carga el autoloader de Composer y el kernel HTTP.
14. Service Container: Laravel resuelve dependencias e instancia los servicios necesarios.
15. Middleware: la solicitud pasa por capas de middleware global y de ruta (autenticación, CSRF, logging).
16. Router: determina qué controlador y método corresponden a la URL y método HTTP.
17. Controller: ejecuta la lógica de negocio, interactúa con modelos y prepara datos.
18. Response: genera la respuesta (vista Blade, JSON, redirección) y la envía al cliente.

Ciclo de vida en Express.js

```
const express = require('express');
const app = express();

// 1. Middleware global (se ejecuta en cada solicitud)
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

// 2. Middleware personalizado (logging)
app.use((req, res, next) => {
  console.log(`${req.method} ${req.url}`);
  next(); // pasa al siguiente middleware o ruta
});

// 3. Ruta → Controlador
```

```
app.get('/tareas', TareaController.index);

// 4. Middleware de errores
app.use((err, req, res, next) => {
  res.status(500).json({ error: err.message });
});

app.listen(3000);
```

3.3 Variables de entorno y configuración del proyecto

Las variables de entorno son valores de configuración que se definen fuera del código fuente y que varían según el entorno de ejecución (desarrollo, pruebas, producción). Su uso es una práctica fundamental en el desarrollo backend moderno porque permite separar la configuración sensible —como credenciales de base de datos, claves API y secretos de cifrado— del código de la aplicación.

3.3.1 Archivos de configuración (.env)

El archivo .env es un archivo de texto plano que contiene pares clave=valor. Se ubica en la raíz del proyecto y no debe incluirse en el control de versiones (se agrega a .gitignore). En su lugar, se incluye un archivo .env.example con las claves pero sin valores sensibles, sirviendo como plantilla para otros desarrolladores.

Archivo .env típico de Laravel

```
APP_NAME=MiAplicacion
APP_ENV=local
APP_KEY=base64:clave_generada_automaticamente
APP_DEBUG=true
APP_URL=http://localhost

DB_CONNECTION=pgsql
DB_HOST=127.0.0.1
DB_PORT=5432
DB_DATABASE=mi_base_datos
DB_USERNAME=postgres
DB_PASSWORD=mi_contraseña_segura

MAIL_MAILER=smtp
MAIL_HOST=smtp.mailtrap.io
MAIL_PORT=2525
```

Archivo .env para Express.js

```
PORT=3000
NODE_ENV=development
DB_HOST=127.0.0.1
DB_PORT=5432
DB_NAME=mi_base_datos
DB_USER=postgres
DB_PASS=mi_contraseña_segura
SESSION_SECRET=clave_secreta_sesiones
JWT_SECRET=clave_secreta_jwt
```

Para acceder a estas variables en el código:

- Laravel: la función env('DB_HOST', 'default') lee la variable; los archivos en config/ (database.php, mail.php) centralizan la configuración.
- Express: el paquete dotenv carga las variables al inicio con require('dotenv').config(), y se acceden con process.env.DB_HOST.

> **ADVERTENCIA**
> Nunca incluyas el archivo .env en el repositorio Git. Contiene credenciales sensibles que podrían comprometer la seguridad de la aplicación. Siempre agrega .env al archivo .gitignore y distribuye .env.example como plantilla.

3.3.2 Gestión de entornos: desarrollo, pruebas y producción

Una aplicación web moderna opera en al menos tres entornos distintos, cada uno con su propia configuración:

Entorno	Variable típica	Características
Desarrollo (local)	APP_ENV=local / NODE_ENV=development	Debug activo, logs detallados, base de datos local, sin caché
Pruebas (testing)	APP_ENV=testing / NODE_ENV=test	BD en memoria o separada, seeds específicos, sin servicios externos reales
Producción	APP_ENV=production / NODE_ENV=production	Debug desactivado, logs mínimos, caché activo, HTTPS obligatorio

Uso condicional del entorno en Laravel

```
// En config/app.php
'debug' => env('APP_DEBUG', false),

// En un controlador
if (app()->environment('production')) {
    // Lógica específica de producción
}
```

Uso condicional del entorno en Express

```
require('dotenv').config();

if (process.env.NODE_ENV === 'development') {
  app.use(morgan('dev')); // Logging detallado
}

if (process.env.NODE_ENV === 'production') {
  app.use(helmet()); // Cabeceras de seguridad
  app.use(compression()); // Compresión gzip
}
```

3.4 Frameworks MVC backend

Un framework backend es un conjunto de herramientas, bibliotecas y convenciones que proporcionan una estructura organizada para desarrollar aplicaciones web del lado del servidor. Los frameworks modernos implementan patrones de diseño probados, automatizan tareas repetitivas y promueven buenas prácticas que aceleran el desarrollo y mejoran la mantenibilidad del código.

3.4.1 Introducción al patrón MVC

El patrón Modelo-Vista-Controlador (MVC) es un patrón arquitectónico que separa una aplicación en tres componentes interconectados, cada uno con una responsabilidad específica:

> **DEFINICIÓN**
> MVC (Model-View-Controller) es un patrón arquitectónico que divide la aplicación en tres capas: el Modelo gestiona los datos y la lógica de negocio, la Vista se encarga de la presentación al usuario, y el Controlador actúa como intermediario que recibe solicitudes, coordina el modelo y selecciona la vista apropiada.

Componente	Responsabilidad	Ejemplo en Laravel	Ejemplo en Express
Modelo	Datos, lógica de negocio, reglas de validación	app/Models/Task.php (Eloquent)	models/Task.js (Sequelize)
Vista	Presentación, interfaz de usuario, plantillas	resources/views/tasks/index.blade.php	views/tasks/index.ejs
Controlador	Recibir solicitudes, coordinar modelo y vista	app/Http/Controllers/TaskController.php	controllers/taskController.js

Las ventajas del patrón MVC incluyen:

- Separación de responsabilidades: cada componente tiene una función clara, facilitando el mantenimiento.
- Reutilización: los modelos pueden ser consumidos por múltiples controladores y las vistas pueden reutilizar componentes.
- Testabilidad: al estar separadas, las capas pueden probarse de forma independiente (pruebas unitarias al modelo, pruebas de integración al controlador).
- Trabajo en equipo: diferentes desarrolladores pueden trabajar simultáneamente en modelos, vistas y controladores.

3.4.2 Laravel: estructura y configuración de un proyecto

Laravel es el framework PHP más popular del ecosistema. Sigue el patrón MVC y ofrece un conjunto completo de herramientas: ORM (Eloquent), motor de plantillas (Blade), sistema de migraciones, autenticación, colas de trabajo y más. La versión de referencia para este curso es Laravel 13.

Para crear un nuevo proyecto Laravel:

Creación de un proyecto Laravel

```
# Instalar el instalador de Laravel (una vez)
composer global require laravel/installer

# Crear proyecto
```

```
laravel new mi-proyecto

# O con Composer directamente
composer create-project laravel/laravel mi-proyecto

# Iniciar servidor de desarrollo
cd mi-proyecto
php artisan serve
```

Estructura de directorios de un proyecto Laravel:

Directorio/Archivo	Propósito
app/Models/	Modelos Eloquent (lógica de datos)
app/Http/Controllers/	Controladores que manejan las solicitudes
app/Http/Middleware/	Middleware para filtrar solicitudes HTTP
routes/web.php	Rutas web (con sesión, CSRF, cookies)
routes/api.php	Rutas API (sin estado, token-based)
resources/views/	Plantillas Blade (.blade.php)
database/migrations/	Migraciones de base de datos
database/seeders/	Seeders para datos de prueba
config/	Archivos de configuración (database.php, app.php)
public/	Document root (index.php, assets estáticos)
.env	Variables de entorno
composer.json	Dependencias PHP

3.4.3 Node.js con Express: estructura y configuración de un proyecto

Express es el framework web más utilizado en el ecosistema Node.js. A diferencia de Laravel, Express es minimalista y no impone una estructura de directorios específica, lo que da libertad al desarrollador pero requiere establecer convenciones propias.

Creación de un proyecto Express

```
# Crear directorio e inicializar
mkdir mi-proyecto && cd mi-proyecto
npm init -y

# Instalar dependencias principales
npm install express dotenv ejs
npm install --save-dev nodemon

# Estructura recomendada:
# mi-proyecto/
#   ├── controllers/
#   ├── models/
#   ├── views/
#   ├── routes/
#   ├── middleware/
#   ├── config/
#   ├── public/
#   ├── app.js
#   ├── .env
#   └── package.json
```

Archivo app.js básico de Express

```
require('dotenv').config();
const express = require('express');
const path = require('path');

const app = express();
const PORT = process.env.PORT || 3000;

// Configurar motor de plantillas
app.set('view engine', 'ejs');
app.set('views', path.join(__dirname, 'views'));

// Middleware
app.use(express.json());
app.use(express.urlencoded({ extended: true }));
app.use(express.static(path.join(__dirname, 'public')));

// Rutas
app.use('/', require('./routes/index'));
app.use('/tareas', require('./routes/tareas'));

app.listen(PORT, () => {
  console.log(`Servidor en http://localhost:${PORT}`);
});
```

3.4.4 Enrutamiento y controladores

El enrutamiento es el mecanismo que asocia URLs y métodos HTTP con funciones o métodos de controladores específicos. Es el punto de entrada para toda la lógica de la aplicación.

Rutas RESTful para un recurso 'tareas':

Método HTTP	URI	Acción	Descripción
GET	/tareas	index	Listar todas las tareas
GET	/tareas/create	create	Mostrar formulario de creación
POST	/tareas	store	Guardar nueva tarea
GET	/tareas/{id}	show	Mostrar una tarea específica
GET	/tareas/{id}/edit	edit	Mostrar formulario de edición
PUT/PATCH	/tareas/{id}	update	Actualizar tarea existente
DELETE	/tareas/{id}	destroy	Eliminar tarea

Rutas en Laravel (routes/web.php)

```
use App\Http\Controllers\TaskController;

// Ruta resource genera las 7 rutas RESTful automáticamente
Route::resource('tareas', TaskController::class);

// Equivalente a definir cada una manualmente:
Route::get('/tareas', [TaskController::class, 'index']);
Route::get('/tareas/create', [TaskController::class, 'create']);
Route::post('/tareas', [TaskController::class, 'store']);
Route::get('/tareas/{tarea}', [TaskController::class, 'show']);
Route::get('/tareas/{tarea}/edit', [TaskController::class, 'edit']);
Route::put('/tareas/{tarea}', [TaskController::class, 'update']);
Route::delete('/tareas/{tarea}', [TaskController::class, 'destroy']);
```

Controlador en Laravel (app/Http/Controllers/TaskController.php)

```
namespace App\Http\Controllers;

use App\Models\Task;
use Illuminate\Http\Request;

class TaskController extends Controller
{
    public function index()
    {
        $tasks = Task::orderBy('created_at', 'desc')->get();
        return view('tasks.index', compact('tasks'));
    }

    public function store(Request $request)
    {
        $validated = $request->validate([
            'title' => 'required|max:255',
            'description' => 'nullable|string',
        ]);
        Task::create($validated);
        return redirect()->route('tareas.index')
                         ->with('success', 'Tarea creada');
    }
}
```

Rutas y controlador en Express

```
// routes/tareas.js
const express = require('express');
const router = express.Router();
const ctrl = require('../controllers/tareaController');

router.get('/', ctrl.index);
router.get('/create', ctrl.create);
router.post('/', ctrl.store);
router.get('/:id', ctrl.show);
router.get('/:id/edit', ctrl.edit);
router.put('/:id', ctrl.update);
router.delete('/:id', ctrl.destroy);

module.exports = router;

// controllers/tareaController.js
const Task = require('../models/Task');

exports.index = async (req, res) => {
  const tasks = await Task.findAll({ order: [['createdAt','DESC']] });
  res.render('tasks/index', { tasks });
};

exports.store = async (req, res) => {
  await Task.create(req.body);
  res.redirect('/tareas');
};
```

3.5 Lógica de negocio y manejo de sesiones

HTTP es un protocolo sin estado (stateless): cada solicitud es independiente y el servidor no recuerda solicitudes anteriores del mismo cliente. Sin embargo, la mayoría de las aplicaciones web necesitan mantener estado entre solicitudes —por ejemplo, saber quién es el usuario autenticado, qué productos tiene en su carrito o qué preferencias ha seleccionado. Las sesiones y cookies son los mecanismos que resuelven esta limitación.

3.5.1 Variables de sesión

DEFINICIÓN

Una sesión es un mecanismo del lado del servidor que permite almacenar datos específicos de un usuario entre múltiples solicitudes HTTP. El servidor genera un identificador único (session ID), lo envía al cliente como cookie, y en solicitudes posteriores utiliza ese ID para recuperar los datos almacenados.

Funcionamiento de las sesiones:

19. El usuario accede a la aplicación por primera vez.
20. El servidor crea una sesión con un ID único (ej. abc123def456) y almacena los datos en el servidor (archivo, base de datos o memoria).
21. El servidor envía el session ID al navegador como una cookie (Set-Cookie: session_id=abc123def456).
22. En solicitudes posteriores, el navegador envía automáticamente la cookie, y el servidor la usa para recuperar los datos de sesión.
23. Al cerrar sesión o expirar el tiempo, el servidor elimina los datos de sesión.

Sesiones en Laravel

```
// Almacenar datos en sesión
$request->session()->put('carrito', $items);
session(['usuario_tema' => 'oscuro']);

// Leer datos de sesión
$carrito = session('carrito', []); // con valor por defecto
$tema = $request->session()->get('usuario_tema');

// Verificar existencia
if ($request->session()->has('carrito')) { ... }

// Eliminar datos
$request->session()->forget('carrito');
$request->session()->flush(); // eliminar todo

// Flash data (solo disponible en la siguiente solicitud)
$request->session()->flash('mensaje', 'Operación exitosa');
```

Sesiones en Express (con express-session)

```
const session = require('express-session');

app.use(session({
  secret: process.env.SESSION_SECRET,
  resave: false,
  saveUninitialized: false,
  cookie: {
    secure: process.env.NODE_ENV === 'production',
    httpOnly: true,
    maxAge: 1000 * 60 * 60 * 24 // 24 horas
  }
}));
```

```
// Almacenar en sesión
req.session.carrito = items;

// Leer de sesión
const carrito = req.session.carrito || [];

// Destruir sesión
req.session.destroy();
```

Driver de sesión	Almacenamiento	Uso recomendado
file	Archivos en storage/framework/sessions	Desarrollo (Laravel por defecto)
database	Tabla sessions en la BD	Producción con múltiples servidores
redis / memcached	Almacén en memoria (key-value)	Alto rendimiento, datos temporales
cookie	Cookie cifrada en el cliente	Datos pequeños, sin servidor de estado

3.5.2 Cookies y autenticación básica

DEFINICIÓN

Una cookie es un pequeño fragmento de datos que el servidor envía al navegador y que este almacena localmente. El navegador incluye automáticamente las cookies en cada solicitud al mismo dominio, permitiendo al servidor identificar al cliente y mantener estado.

Atributos importantes de las cookies:

Atributo	Descripción	Valor recomendado
HttpOnly	Impide acceso desde JavaScript (XSS)	true (siempre para session IDs)
Secure	Solo se envía sobre HTTPS	true en producción
SameSite	Controla envío en solicitudes cross-site	Lax (navegación) o Strict (máxima protección)
Max-Age / Expires	Tiempo de vida de la cookie	Según necesidad (sesión o persistente)
Domain / Path	Alcance de la cookie	Restringir al dominio/ruta necesario

Autenticación básica con sesiones:

La autenticación basada en sesiones sigue este flujo: el usuario envía sus credenciales (email y contraseña), el servidor las verifica contra la base de datos, y si son válidas, crea una sesión con los datos del usuario autenticado. En cada solicitud posterior, el middleware verifica que exista una sesión válida.

Autenticación con sesiones en Laravel

```
// Login
public function login(Request $request)
{
    $credentials = $request->validate([
        'email' => 'required|email',
        'password' => 'required',
    ]);

    if (Auth::attempt($credentials)) {
```

```
        $request->session()->regenerate();
        return redirect()->intended('dashboard');
    }

    return back()->withErrors([
        'email' => 'Credenciales inválidas.',
    ]);
}

// Proteger rutas con middleware
Route::middleware('auth')->group(function () {
    Route::resource('tareas', TaskController::class);
});
```

NOTA

Laravel 13 incluye el atributo #[Middleware] que permite declarar middleware directamente en el controlador con PHP Attributes, como alternativa a definirlos en las rutas. Esto mejora la legibilidad y el encapsulamiento de la lógica de autorización.

3.6 Motor de plantillas

Un motor de plantillas (template engine) es un componente que permite generar HTML dinámico en el servidor combinando plantillas estáticas con datos proporcionados por los controladores. La idea central es separar la lógica de presentación del código de la aplicación: el controlador prepara los datos y el motor de plantillas se encarga de incrustarlos en el HTML.

Los motores de plantillas ofrecen:

- Interpolación de variables: insertar valores dinámicos en el HTML.
- Estructuras de control: condicionales (if/else), bucles (for/foreach).
- Herencia de plantillas (layouts): definir una plantilla base que las vistas hijas extienden.
- Componentes y parciales: fragmentos reutilizables de HTML.
- Escape automático: prevención de XSS al escapar la salida por defecto.

3.6.1 Blade (Laravel): renderizado de vistas desde el servidor

Blade es el motor de plantillas de Laravel. Sus archivos usan la extensión .blade.php y se almacenan en resources/views/. Blade compila las plantillas a PHP puro y las cachea, por lo que no añade penalización de rendimiento.

Layout base en Blade (resources/views/layouts/app.blade.php)

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>@yield('title', 'Mi App')</title>
    <link rel="stylesheet" href="{{ asset('css/app.css') }}">
</head>
<body>
    <nav>
        @auth
            <span>{{ Auth::user()->name }}</span>
            <form action="/logout" method="POST">
                @csrf
                <button type="submit">Salir</button>
            </form>
        @endauth
    </nav>

    <main>
        @if(session('success'))
            <div class="alert">{{ session('success') }}</div>
        @endif

        @yield('content')
    </main>
</body>
</html>
```

Vista hija que extiende el layout (resources/views/tasks/index.blade.php)

```
@extends('layouts.app')

@section('title', 'Lista de Tareas')

@section('content')
    <h1>Mis Tareas</h1>

    @forelse($tasks as $task)
        <div class="task-card">
```

```
            <h3>{{ $task->title }}</h3>
            <p>{{ $task->description }}</p>
            <span>{{ $task->created_at->diffForHumans() }}</span>
        </div>
    @empty
        <p>No tienes tareas registradas.</p>
    @endforelse
@endsection
```

Directivas principales de Blade:

Directiva	Función	Ejemplo
{{ $var }}	Mostrar variable (con escape XSS)	{{ $task->title }}
{!! $var !!}	Mostrar sin escape (HTML raw)	Solo para HTML confiable
@if / @elseif / @else	Condicionales	@if($tasks->isEmpty()) ...
@foreach / @forelse	Bucles	@foreach($tasks as $task) ...
@extends / @section / @yield	Herencia de layouts	@extends('layouts.app')
@include	Incluir parcial	@include('partials.alert')
@csrf	Token CSRF en formularios	<form> @csrf ... </form>
@method('PUT')	Simular PUT/PATCH/DELETE	@method('DELETE')

3.6.2 EJS / Handlebars (Node.js): uso de plantillas dinámicas

En el ecosistema Node.js existen varios motores de plantillas. Los dos más populares son EJS (Embedded JavaScript) y Handlebars. EJS permite incrustar JavaScript directamente en el HTML, mientras que Handlebars usa una sintaxis más restrictiva basada en expresiones {{}}.

Plantilla EJS (views/tasks/index.ejs)

```
<!DOCTYPE html>
<html lang="es">
<head>
    <title>Lista de Tareas</title>
</head>
<body>
    <h1>Mis Tareas</h1>

    <% if (tasks.length === 0) { %>
        <p>No tienes tareas registradas.</p>
    <% } else { %>
        <% tasks.forEach(task => { %>
            <div class="task-card">
                <h3><%= task.title %></h3>
                <p><%= task.description %></p>
            </div>
        <% }) %>
    <% } %>
</body>
</html>
```

Sintaxis EJS	Función
<%= var %>	Mostrar variable con escape HTML (seguro)
<%- var %>	Mostrar sin escape (HTML raw)
<% código %>	Ejecutar código JavaScript (sin salida)
<%- include('header') %>	Incluir parcial

Plantilla Handlebars (views/tasks/index.hbs)

```
<h1>Mis Tareas</h1>

{{#if tasks.length}}
    {{#each tasks}}
        <div class="task-card">
            <h3>{{this.title}}</h3>
            <p>{{this.description}}</p>
        </div>
    {{/each}}
{{else}}
    <p>No tienes tareas registradas.</p>
{{/if}}
```

NOTA

En este curso se utiliza EJS como motor de plantillas para Express por su similitud con HTML y la posibilidad de usar JavaScript directamente, lo que reduce la curva de aprendizaje para estudiantes que ya dominan JS. Blade se utiliza en Laravel.

3.7 Integración con bases de datos

La integración con bases de datos es una de las tareas fundamentales del desarrollo backend. Las aplicaciones web necesitan persistir datos —usuarios, tareas, pedidos, productos— y consultarlos de forma eficiente. Los frameworks modernos proporcionan herramientas que simplifican esta integración mediante ORM (Object-Relational Mapping), migraciones y seeders.

3.7.1 Conexión de la aplicación a la base de datos

La conexión se configura a través de las variables de entorno y los archivos de configuración del framework. El sistema de base de datos para este curso es PostgreSQL.

Configuración de la conexión en Laravel (config/database.php + .env)

```
// .env
DB_CONNECTION=pgsql
DB_HOST=127.0.0.1
DB_PORT=5432
DB_DATABASE=tareas_db
DB_USERNAME=postgres
DB_PASSWORD=password

// config/database.php (fragmento relevante)
'pgsql' => [
    'driver' => 'pgsql',
    'host' => env('DB_HOST', '127.0.0.1'),
    'port' => env('DB_PORT', '5432'),
    'database' => env('DB_DATABASE', 'forge'),
    'username' => env('DB_USERNAME', 'forge'),
    'password' => env('DB_PASSWORD', ''),
],
```

Configuración de la conexión en Express (con Sequelize)

```
// config/database.js
const { Sequelize } = require('sequelize');

const sequelize = new Sequelize(
  process.env.DB_NAME,
  process.env.DB_USER,
  process.env.DB_PASS,
  {
    host: process.env.DB_HOST,
    port: process.env.DB_PORT,
    dialect: 'postgres',
    logging: process.env.NODE_ENV === 'development'
             ? console.log : false,
  }
);

module.exports = sequelize;
```

3.7.2 ORM: concepto y ventajas

DEFINICIÓN

Un ORM (Object-Relational Mapping) es una técnica que permite interactuar con la base de datos relacional utilizando objetos del lenguaje de programación en lugar de escribir SQL directamente. Cada tabla de la base de datos se mapea a una clase (modelo), cada fila a una instancia, y cada columna a una propiedad del objeto.

Ventajas del ORM:

- Abstracción del SQL: el desarrollador trabaja con objetos y métodos, no con cadenas SQL.
- Portabilidad: cambiar de motor de BD (PostgreSQL, MySQL, SQLite) requiere cambiar solo la configuración, no el código.
- Seguridad: previene inyección SQL al parametrizar las consultas automáticamente.
- Productividad: operaciones CRUD se reducen a una línea de código (Task::create(), Task.findAll()).
- Relaciones: permite definir y navegar relaciones entre tablas (hasMany, belongsTo) de forma intuitiva.

> **ADVERTENCIA**
> Aunque el ORM simplifica el desarrollo, es importante comprender SQL subyacente. En consultas complejas o de alto rendimiento, el ORM puede generar queries ineficientes (N+1 problem). Herramientas como Laravel Debugbar o Sequelize logging permiten inspeccionar las queries generadas.

3.7.3 Eloquent ORM (Laravel) y Sequelize (Node.js)

Eloquent es el ORM de Laravel. Cada modelo Eloquent extiende la clase Model y se asocia a una tabla de la base de datos. Sequelize cumple el mismo rol en el ecosistema Node.js.

Modelo Eloquent (app/Models/Task.php)

```php
namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Task extends Model
{
    // Campos que se pueden asignar masivamente
    protected $fillable = ['title', 'description', 'completed'];

    // Casts automáticos de tipos
    protected $casts = [
        'completed' => 'boolean',
    ];

    // Relación: una tarea pertenece a un usuario
    public function user()
    {
        return $this->belongsTo(User::class);
    }
}
```

Modelo Sequelize (models/Task.js)

```javascript
const { DataTypes } = require('sequelize');
const sequelize = require('../config/database');

const Task = sequelize.define('Task', {
  id: {
    type: DataTypes.UUID,
    defaultValue: DataTypes.UUIDV4,
    primaryKey: true,
  },
  title: {
    type: DataTypes.STRING,
    allowNull: false,
    validate: { notEmpty: true, len: [1, 255] },
  },
  description: {
```

```
    type: DataTypes.TEXT,
    allowNull: true,
  },
  completed: {
    type: DataTypes.BOOLEAN,
    defaultValue: false,
  },
}, {
  tableName: 'tasks',
  timestamps: true,
});

module.exports = Task;
```

Operaciones CRUD comparadas:

Operación	Eloquent (Laravel)	Sequelize (Node.js)
Crear	Task::create(['title' => 'Nueva'])	Task.create({ title: 'Nueva' })
Leer todos	Task::all() o Task::get()	Task.findAll()
Leer uno	Task::find($id) o Task::findOrFail($id)	Task.findByPk(id)
Actualizar	$task->update(['title' => 'Editada'])	task.update({ title: 'Editada' })
Eliminar	$task->delete()	task.destroy()
Filtrar	Task::where('completed', true)->get()	Task.findAll({ where: { completed: true } })

3.7.4 Migraciones y seeders

Las migraciones son archivos que describen cambios en la estructura de la base de datos (crear tablas, agregar columnas, índices) de forma versionada y reproducible. Funcionan como un control de versiones para el esquema de la BD.

> **DEFINICIÓN**
> Una migración es un archivo de código que define cambios en el esquema de la base de datos. Cada migración tiene dos métodos: up() para aplicar el cambio y down() para revertirlo. Las migraciones se ejecutan en orden cronológico, garantizando que todos los entornos tengan el mismo esquema.

Migración en Laravel (database/migrations/2025_01_01_create_tasks_table.php)

```
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('tasks', function (Blueprint $table) {
            $table->id();
            $table->foreignId('user_id')->constrained()->onDelete('cascade');
            $table->string('title');
            $table->text('description')->nullable();
            $table->boolean('completed')->default(false);
            $table->timestamps();
        });
    }

    public function down(): void
```

```
    {
        Schema::dropIfExists('tasks');
    }
};
```

Migración en Sequelize (migrations/20250101-create-tasks.js)

```
'use strict';

module.exports = {
  async up(queryInterface, Sequelize) {
    await queryInterface.createTable('tasks', {
      id: {
        type: Sequelize.UUID,
        defaultValue: Sequelize.UUIDV4,
        primaryKey: true,
      },
      user_id: {
        type: Sequelize.UUID,
        references: { model: 'users', key: 'id' },
        onDelete: 'CASCADE',
      },
      title: { type: Sequelize.STRING, allowNull: false },
      description: { type: Sequelize.TEXT },
      completed: { type: Sequelize.BOOLEAN, defaultValue: false },
      createdAt: { type: Sequelize.DATE },
      updatedAt: { type: Sequelize.DATE },
    });
  },
  async down(queryInterface) {
    await queryInterface.dropTable('tasks');
  },
};
```

Los seeders son archivos que insertan datos iniciales o de prueba en la base de datos. Son útiles para:

- Poblar la base de datos de desarrollo con datos realistas para probar la aplicación.
- Insertar datos base necesarios (roles, categorías, configuraciones).
- Crear escenarios de prueba reproducibles.

Seeder en Laravel (database/seeders/TaskSeeder.php)

```
namespace Database\Seeders;

use App\Models\Task;
use Illuminate\Database\Seeder;

class TaskSeeder extends Seeder
{
    public function run(): void
    {
        Task::factory(50)->create();
    }
}

// Ejecutar: php artisan db:seed --class=TaskSeeder
```

Comandos principales de migraciones:

Acción	Laravel	Sequelize CLI
Ejecutar migraciones	php artisan migrate	npx sequelize-cli db:migrate
Revertir última	php artisan migrate:rollback	npx sequelize-cli db:migrate:undo

Revertir todas	php artisan migrate:reset	npx sequelize-cli db:migrate:undo:all
Recrear todo + seed	php artisan migrate:fresh --seed	Reset + seed manual
Crear migración	php artisan make:migration create_X_table	npx sequelize-cli migration:generate --name X
Ejecutar seeders	php artisan db:seed	npx sequelize-cli db:seed:all

3.7.5 Relaciones entre modelos

Las relaciones entre modelos reflejan las asociaciones entre tablas en la base de datos. Los ORM proporcionan métodos para definir estas relaciones de forma declarativa, permitiendo navegar entre entidades relacionadas sin escribir joins SQL manualmente.

Tipo de relación	Descripción	Ejemplo
Uno a muchos (1:N)	Un registro tiene muchos relacionados	Un usuario tiene muchas tareas
Muchos a uno (N:1)	Muchos registros pertenecen a uno	Muchas tareas pertenecen a un usuario
Muchos a muchos (N:M)	Requiere tabla pivote intermedia	Tareas tienen muchas etiquetas y viceversa
Uno a uno (1:1)	Un registro tiene exactamente uno relacionado	Un usuario tiene un perfil

Relaciones en Eloquent (Laravel)

```
// app/Models/User.php
class User extends Authenticatable
{
    public function tasks()
    {
        return $this->hasMany(Task::class);
    }

    public function profile()
    {
        return $this->hasOne(Profile::class);
    }
}

// app/Models/Task.php
class Task extends Model
{
    public function user()
    {
        return $this->belongsTo(User::class);
    }

    public function tags()
    {
        return $this->belongsToMany(Tag::class);
    }
}

// Uso en controlador
$user = User::with('tasks')->find(1); // Eager loading
$tasks = $user->tasks; // Acceso a la colección
$task->user->name;     // Acceso inverso
```

Relaciones en Sequelize (Node.js)

```
// models/associations.js
const User = require('./User');
const Task = require('./Task');
const Tag  = require('./Tag');

// Uno a muchos
User.hasMany(Task, { foreignKey: 'user_id' });
Task.belongsTo(User, { foreignKey: 'user_id' });

// Muchos a muchos (requiere tabla pivote task_tags)
Task.belongsToMany(Tag, { through: 'task_tags' });
Tag.belongsToMany(Task, { through: 'task_tags' });

// Uso con eager loading
const user = await User.findByPk(1, {
  include: [{ model: Task }]
});
const tasks = user.Tasks;
```

ADVERTENCIA

El problema N+1 ocurre cuando se ejecuta una consulta por cada elemento de una colección. Si se tienen 100 usuarios y se accede a user.tasks para cada uno, se ejecutan 101 queries (1 para usuarios + 100 para tareas). La solución es usar eager loading: User::with('tasks')->get() en Eloquent o include en Sequelize, que resuelve todo en 2 queries.

3.8 Patrones de diseño aplicados al backend

A medida que las aplicaciones crecen en complejidad, organizar el código de manera estructurada se vuelve crucial. Los patrones de diseño son soluciones probadas y reutilizables para problemas comunes de diseño de software. En el desarrollo backend, dos patrones destacan por su capacidad de mantener el código limpio, testeable y mantenible: el patrón Repository y el patrón Service Layer.

3.8.1 Patrón Repository

DEFINICIÓN

El patrón Repository actúa como una capa de abstracción entre la lógica de negocio y la capa de acceso a datos. Encapsula las operaciones de consulta y persistencia en una clase dedicada, de modo que los controladores y servicios no necesitan conocer los detalles de implementación del ORM o la base de datos.

Sin el patrón Repository, los controladores contienen directamente las consultas Eloquent o Sequelize. Esto genera acoplamiento entre la lógica de negocio y la capa de datos, dificultando las pruebas unitarias y los cambios de implementación.

Repository en Laravel

```
// app/Repositories/TaskRepositoryInterface.php
interface TaskRepositoryInterface
{
    public function all();
    public function find(int $id);
    public function create(array $data);
    public function update(int $id, array $data);
    public function delete(int $id);
    public function findByUser(int $userId);
}

// app/Repositories/EloquentTaskRepository.php
class EloquentTaskRepository implements TaskRepositoryInterface
{
    public function all()
    {
        return Task::orderBy('created_at', 'desc')->get();
    }

    public function find(int $id)
    {
        return Task::findOrFail($id);
    }

    public function create(array $data)
    {
        return Task::create($data);
    }

    public function findByUser(int $userId)
    {
        return Task::where('user_id', $userId)->get();
    }
    // ... update, delete
}
```

Uso del Repository en el controlador

```
class TaskController extends Controller
{
    public function __construct(
```

```
        private TaskRepositoryInterface $tasks
    ) {}

    public function index()
    {
        return view('tasks.index', [
            'tasks' => $this->tasks->all()
        ]);
    }

    public function store(Request $request)
    {
        $this->tasks->create($request->validated());
        return redirect()->route('tareas.index');
    }
}
```

3.8.2 Service Layer

DEFINICIÓN

El patrón Service Layer (capa de servicio) encapsula la lógica de negocio compleja en clases dedicadas, separándola de los controladores. Los servicios coordinan operaciones que involucran múltiples modelos, repositorios o servicios externos, manteniendo los controladores delgados y enfocados en manejar solicitudes y respuestas.

Service Layer en Laravel

```
// app/Services/TaskService.php
class TaskService
{
    public function __construct(
        private TaskRepositoryInterface $taskRepo,
        private NotificationService $notifier
    ) {}

    public function createTask(array $data, User $user): Task
    {
        $data['user_id'] = $user->id;
        $task = $this->taskRepo->create($data);

        // Lógica de negocio adicional
        if ($user->tasks()->count() >= 100) {
            $this->notifier->send($user, 'Tienes 100+ tareas');
        }

        return $task;
    }

    public function completeTask(int $id): Task
    {
        $task = $this->taskRepo->find($id);
        $this->taskRepo->update($id, ['completed' => true]);
        return $task->fresh();
    }
}

// Controlador simplificado
class TaskController extends Controller
{
    public function __construct(
        private TaskService $taskService
    ) {}

    public function store(Request $request)
    {
```

```
        $this->taskService->createTask(
            $request->validated(),
            $request->user()
        );
        return redirect()->route('tareas.index');
    }
}
```

La arquitectura resultante con ambos patrones sigue esta jerarquía:

24. Controlador → recibe la solicitud, valida datos, delega al servicio.
25. Servicio → ejecuta la lógica de negocio, coordina repositorios y otros servicios.
26. Repositorio → accede a la base de datos a través del ORM.
27. Modelo → representa la entidad de datos (tabla).

> **NOTA**
> En aplicaciones pequeñas, usar directamente el ORM en los controladores es aceptable. Los patrones Repository y Service Layer se vuelven valiosos a medida que la aplicación crece y la lógica de negocio se vuelve más compleja. La clave es saber cuándo aplicarlos, evitando sobre-ingeniería prematura.

3.9 CRUD completo

CRUD es el acrónimo de las cuatro operaciones fundamentales sobre datos persistentes: Create (crear), Read (leer), Update (actualizar) y Delete (eliminar). Implementar un CRUD completo significa integrar todas las capas de la aplicación —rutas, controladores, modelos, vistas y base de datos— para permitir al usuario gestionar un recurso de principio a fin.

Operación CRUD	Método HTTP	Acción del controlador	Vista asociada
Create	GET + POST	create() + store()	Formulario de creación
Read	GET	index() + show()	Listado y detalle
Update	GET + PUT/PATCH	edit() + update()	Formulario de edición
Delete	DELETE	destroy()	Confirmación (modal o formulario)

3.9.1 Integración de vistas, lógica y base de datos

Un CRUD completo integra tres capas en cada operación:

28. Vista (frontend): formularios HTML para la entrada de datos y tablas/listas para la visualización.
29. Controlador (lógica): recibe los datos del formulario, los valida, interactúa con el modelo y decide qué vista mostrar.
30. Modelo + BD (persistencia): el modelo (Eloquent/Sequelize) ejecuta las operaciones SQL subyacentes.

CRUD completo en Laravel — Controlador

```
class TaskController extends Controller
{
    public function index()
    {
        $tasks = Task::where('user_id', auth()->id())
                     ->orderBy('created_at', 'desc')
                     ->paginate(10);
        return view('tasks.index', compact('tasks'));
    }

    public function create()
    {
        return view('tasks.create');
    }

    public function store(Request $request)
    {
        $validated = $request->validate([
            'title'       => 'required|string|max:255',
            'description' => 'nullable|string',
        ]);
        $validated['user_id'] = auth()->id();
        Task::create($validated);
        return redirect()->route('tareas.index')
                         ->with('success', 'Tarea creada');
    }

    public function edit(Task $task)
    {
        $this->authorize('update', $task);
        return view('tasks.edit', compact('task'));
    }
```

```
    public function update(Request $request, Task $task)
    {
        $this->authorize('update', $task);
        $task->update($request->validate([
            'title'       => 'required|string|max:255',
            'description' => 'nullable|string',
            'completed'   => 'boolean',
        ]));
        return redirect()->route('tareas.index')
                         ->with('success', 'Tarea actualizada');
    }

    public function destroy(Task $task)
    {
        $this->authorize('delete', $task);
        $task->delete();
        return redirect()->route('tareas.index')
                         ->with('success', 'Tarea eliminada');
    }
}
```

3.9.2 Validación y manejo de errores

La validación es el proceso de verificar que los datos recibidos del cliente cumplen con las reglas de negocio antes de procesarlos o almacenarlos. Una validación robusta previene datos inconsistentes, inyecciones y errores de integridad.

Estrategias de validación:

- Validación en el cliente (frontend): HTML5 (required, pattern, maxlength) + JavaScript. Es una primera línea de defensa pero nunca debe ser la única.
- Validación en el servidor (backend): la validación definitiva. Siempre se debe validar en el servidor aunque exista validación frontend.
- Validación en el modelo/BD: restricciones de base de datos (NOT NULL, UNIQUE, CHECK) como última red de seguridad.

Validación en Laravel con Form Request

```
// app/Http/Requests/StoreTaskRequest.php
class StoreTaskRequest extends FormRequest
{
    public function authorize(): bool
    {
        return true; // o lógica de autorización
    }

    public function rules(): array
    {
        return [
            'title'       => 'required|string|max:255|min:3',
            'description' => 'nullable|string|max:1000',
            'priority'    => 'in:low,medium,high',
            'due_date'    => 'nullable|date|after:today',
        ];
    }

    public function messages(): array
    {
        return [
            'title.required' => 'El título es obligatorio.',
            'title.max'      => 'El título no puede exceder 255 caracteres.',
            'due_date.after' => 'La fecha debe ser posterior a hoy.',
        ];
    }
```

```
}

// Uso en el controlador (automático)
public function store(StoreTaskRequest $request) {
    Task::create($request->validated());
    // Si falla, Laravel redirige automáticamente con errores
}
```

Validación en Express (con express-validator)

```
const { body, validationResult } = require('express-validator');

// Middleware de validación en las rutas
router.post('/', [
  body('title')
    .trim()
    .notEmpty().withMessage('El título es obligatorio')
    .isLength({ max: 255 }).withMessage('Máximo 255 caracteres'),
  body('description')
    .optional()
    .isLength({ max: 1000 }),
], ctrl.store);

// En el controlador
exports.store = async (req, res) => {
  const errors = validationResult(req);
  if (!errors.isEmpty()) {
    return res.render('tasks/create', {
      errors: errors.array(),
      old: req.body
    });
  }
  await Task.create(req.body);
  res.redirect('/tareas');
};
```

Manejo de errores:

Una aplicación robusta debe capturar y manejar errores de forma elegante, mostrando mensajes útiles al usuario sin exponer detalles internos del sistema. Los frameworks proporcionan mecanismos centralizados para gestionar excepciones.

Código HTTP	Significado	Cuándo usar
200	OK	Solicitud exitosa
201	Created	Recurso creado exitosamente
301/302	Redirect	Redirección permanente/temporal
400	Bad Request	Datos de entrada inválidos
401	Unauthorized	No autenticado
403	Forbidden	Autenticado pero sin permisos
404	Not Found	Recurso no encontrado
422	Unprocessable Entity	Error de validación (APIs)
500	Internal Server Error	Error del servidor

3.9.3 Logging y depuración de aplicaciones

El logging (registro de eventos) es la práctica de registrar información relevante sobre la ejecución de la aplicación: errores, advertencias, accesos, operaciones realizadas. Un buen sistema de logging es esencial para diagnosticar problemas en producción, auditar acciones y entender el comportamiento de la aplicación.

Niveles de logging (según RFC 5424):

Nivel	Descripción	Ejemplo de uso
emergency	Sistema inutilizable	Base de datos completamente caída
alert	Acción inmediata requerida	Disco lleno, certificado expirado
critical	Condición crítica	Excepción no capturada, componente inaccesible
error	Error que necesita atención	Fallo al procesar un pago
warning	Situación anormal pero no crítica	Uso de API deprecada
notice	Evento normal pero significativo	Login exitoso, cambio de contraseña
info	Información general	Servidor iniciado, migración ejecutada
debug	Información detallada para desarrollo	Contenido de variables, queries SQL

Logging en Laravel

```
use Illuminate\Support\Facades\Log;

// Diferentes niveles
Log::info('Usuario logueado', ['user_id' => $user->id]);
Log::warning('Intento de acceso sin permisos', [
    'user' => auth()->id(),
    'url'  => request()->url(),
]);
Log::error('Error al procesar pago', [
    'exception' => $e->getMessage(),
    'order_id'  => $order->id,
]);

// Configuración en config/logging.php
// Canal 'daily' crea un archivo por día en storage/logs/
// Canal 'stack' combina múltiples canales
```

Logging en Express (con Winston)

```
const winston = require('winston');

const logger = winston.createLogger({
  level: process.env.LOG_LEVEL || 'info',
  format: winston.format.combine(
    winston.format.timestamp(),
    winston.format.json()
  ),
  transports: [
    new winston.transports.File({
      filename: 'logs/error.log',
      level: 'error'
    }),
    new winston.transports.File({
      filename: 'logs/combined.log'
    }),
  ],
});
```

```
// En desarrollo, agregar salida a consola
if (process.env.NODE_ENV !== 'production') {
  logger.add(new winston.transports.Console({
    format: winston.format.simple(),
  }));
}

module.exports = logger;
```

Herramientas de depuración:

Herramienta	Framework	Función
Laravel Debugbar	Laravel	Barra de depuración con queries, tiempo, memoria, rutas
Telescope	Laravel	Dashboard completo de solicitudes, excepciones, logs, jobs
dd() / dump()	Laravel	Imprimir variables y detener ejecución
morgan	Express	Middleware de logging HTTP (método, URL, status, tiempo)
debug (npm)	Node.js	Logging condicional por módulo (DEBUG=app:*)
console.log / console.table	Node.js	Depuración básica en consola
Node --inspect	Node.js	Depuración con Chrome DevTools (breakpoints, stepping)

NOTA

En producción, nunca dejes activos dd(), console.log() ni APP_DEBUG=true. Los logs de producción deben registrar solo información relevante (errores, accesos sospechosos) sin exponer datos sensibles como contraseñas o tokens. Utiliza variables de entorno para controlar el nivel de logging según el entorno.